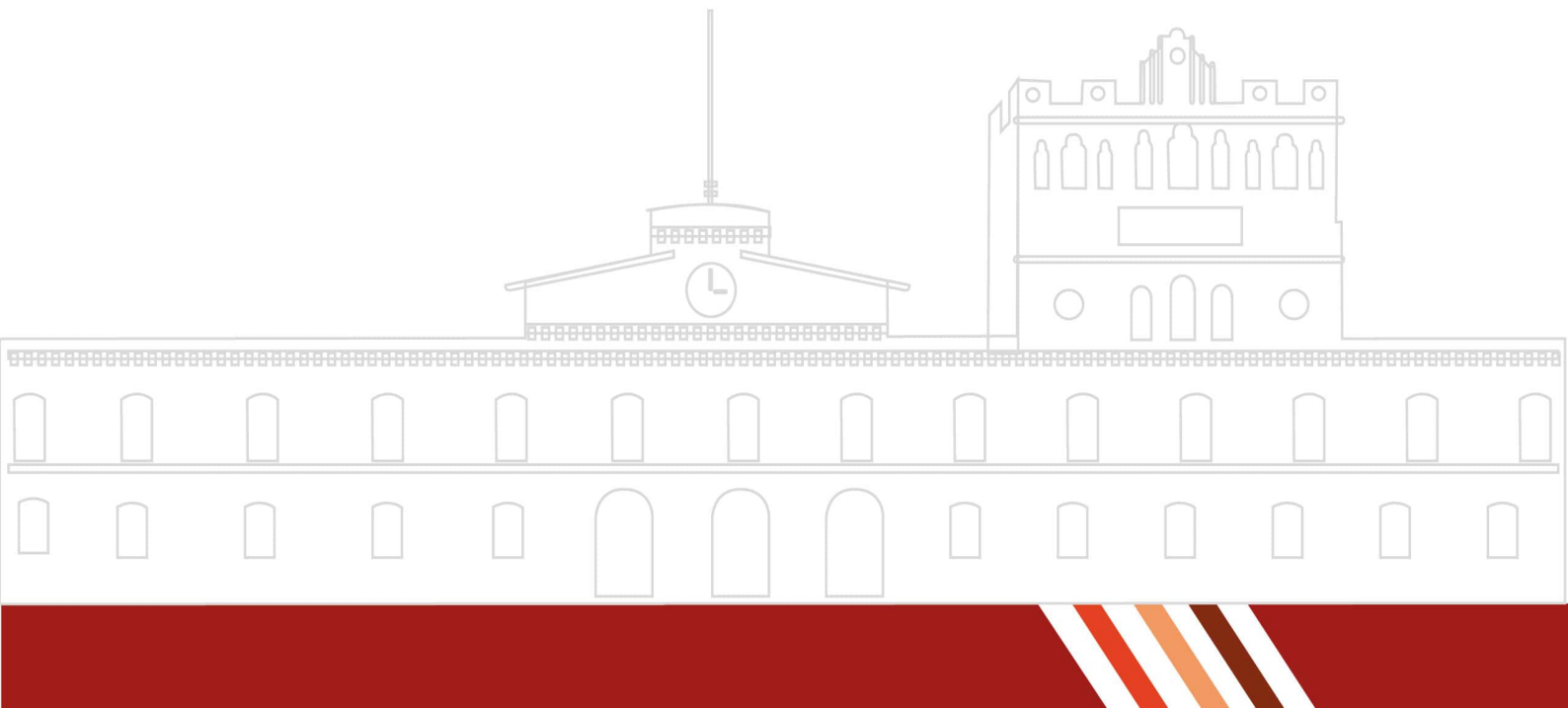


REPORTE DE PROYECTO

Analizador semantico

Materia: Automatas y compiladores

Fecha: 13 de mayo de 2026



1 TEORÍA: Profundidad del Análisis Semántico

El diseño de compiladores es una de las áreas más maduras de las ciencias computacionales, y el análisis semántico representa el puente crítico entre el *front-end* (que entiende el texto) y el *back-end* (que genera el código máquina). Mientras que la sintaxis se describe mediante Gramáticas Libres de Contexto (CFG), la semántica estática de los lenguajes de programación modernos es **sensible al contexto**. Esto significa que no podemos verificar las reglas de un programa simplemente viendo su estructura; necesitamos conocer la “historia” y el “entorno” de cada variable y función.

1.1. Semántica Estática vs. Semántica Dinámica

Es crucial en la teoría de lenguajes distinguir entre lo que el compilador puede hacer y lo que ocurre en tiempo de ejecución:

- **Semántica Estática:** Son las reglas que se pueden comprobar en tiempo de compilación. Esto incluye la verificación de tipos, asegurar que los identificadores estén declarados antes de usarse, y comprobar que las funciones reciban el número correcto de argumentos. De esto se encarga el Analizador Semántico.
- **Semántica Dinámica:** Define el comportamiento del programa en tiempo de ejecución (ej. división por cero, desbordamiento de búfer). El analizador semántico no puede atrapar estos errores, aunque lenguajes modernos (como Rust) intentan mover comprobaciones dinámicas a la fase estática mediante sistemas de tipos afines.

1.2. Teoría de Tipos y Seguridad (Type Safety)

El núcleo del análisis semántico es la **verificación de tipos**.

- **Sistemas Fuertes vs. Débiles:** Un analizador estricto (Java, Haskell) prohibirá operaciones entre tipos incompatibles a menos que exista una conversión explícita.
- **Sistemas Estáticos vs. Dinámicos:** En lenguajes estáticos (C++), el analizador asigna tipos de manera inmutable. En dinámicos (Python), la verificación se delega a la Máquina Virtual.

“Un lenguaje de programación está fuertemente tipado si los errores de tipo son siempre detectados. El objetivo principal de un comprobador de tipos estático es detectar la mayor cantidad posible de estos errores durante el análisis semántico...”

— Scott, M. L. (2015). *Programming Language Pragmatics*. Morgan Kaufmann.

2 HERRAMIENTAS Y TÉCNICAS: La Ingeniería Interna

El analizador semántico no trabaja en el vacío. Recibe un **Árbol de Sintaxis Abstracta (AST)** y lo transforma en un **AST Decorado**.

2.1. La Tabla de Símbolos

Esta es la estructura de datos más importante. Es una estructura dinámica que debe manejar el concepto de **Ámbitos (Scopes)**. Generalmente se implementa como una **pila de tablas Hash** (Stack of Hash Tables). Cuando el analizador entra en un nuevo bloque de código, hace un *push* de una nueva tabla Hash. Cuando sale, hace un *pop*. Por cada identificador, la tabla guarda: nombre, tipo de dato, ubicación de memoria, nivel de anidamiento y firma (si es función).

2.2. Traducción Dirigida por Sintaxis (TDS) y Gramáticas Atribuidas

Formalizado por Donald Knuth, permite asociar reglas de evaluación a la gramática.

Atributo	Flujo de Información	Uso Típico
Sintetizados	De los hijos hacia el padre (Abajo → Arriba).	Evaluar el tipo resultante de una expresión matemática (ej. <code>int + float = float</code>).
Heredados	Del padre hacia los hijos (Arriba → Abajo).	Pasar información del entorno, como heredar el tipo en una declaración múltiple.

Tabla 1: Tipos de Atributos en Gramáticas Atribuidas

“Las gramáticas atribuidas proporcionan un formalismo elegante para especificar la semántica estática y sirven como base para la implementación de la TDS.”

— Aho, A. V., et al. (2006). *Compiladores: principios, técnicas y herramientas*. Pearson.

3 MANEJO DE ERRORES: Resiliencia en la Compilación

Un compilador moderno debe reportar todos los errores posibles de una sola pasada y brindar mensajes claros, sugiriendo soluciones.

3.1. Categorización de Errores Semánticos

- **Resolución de nombres:** Variables no declaradas o declaradas múltiples veces.
- **Tipificación:** Asignaciones inválidas o tipos incompatibles en operadores.
- **Control de flujo:** Uso de `break` fuera de un ciclo.
- **Encapsulamiento:** Intentar acceder a métodos `private` desde fuera de la clase.

3.2. Técnicas de Recuperación: El Patrón “Tipo Error” (Poisoning)

El mayor desafío son los **errores en cascada**. Si el usuario olvida declarar la variable `y` en `x = y + 5;`, el compilador reportaría múltiples errores por la misma omisión. La solución es el *Poisoning*: cuando no se encuentra `y`, se reporta el error y se inyecta en la tabla de símbolos con un tipo especial `Type.ERROR`. Cualquier operación con este tipo arroja silenciosamente `Type.ERROR`, permitiendo continuar el análisis de forma limpia.

4 CÓMO SE CONSTRUYE: Arquitectura de Software

La implementación práctica es un ejercicio de arquitectura de software, usando múltiples recorridos sobre el AST.

4.1. La Arquitectura Multipasada (Multi-pass)

1. **Resolución de Símbolos:** Recorre el AST para leer declaraciones globales y llenar la Tabla de Símbolos.
2. **Análisis de Cuerpos:** Recorre el interior de las funciones, creando las tablas locales.
3. **Comprobación de Tipos:** Evalúa expresiones matemáticas y “decora” el AST.

4.2. El Patrón de Diseño “Visitor”

En lugar de poner la lógica semántica dentro de cada clase de nodo del AST (lo que viola el principio de Responsabilidad Única), se crea una interfaz `Visitor` con métodos como `visit(AssignmentNode)`. El analizador es una clase `SemanticVisitor` que implementa esta interfaz, evaluando tipos recursivamente y aplicando reglas o lanzando errores de forma modularizada.

Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compiladores: principios, técnicas y herramientas* (2ª ed.). Pearson Educación.
- [2] Scott, M. L. (2015). *Programming Language Pragmatics* (4ª ed.). Morgan Kaufmann.
- [3] Appel, A. W. (2002). *Modern Compiler Implementation in Java* (2ª ed.). Cambridge University Press.
- [4] Loudon, K. C. (1997). *Compiler Construction: Principles and Practice*. Cengage Learning.
- [5] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.